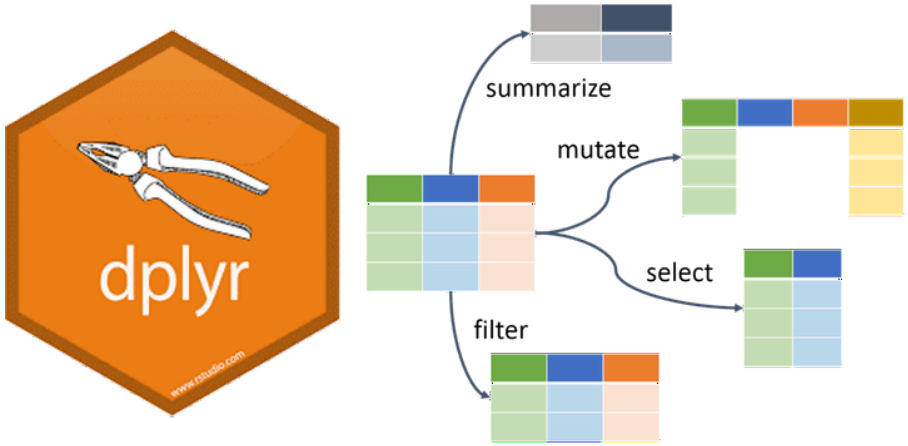
Data Manipulation: dplyr

Pablo E. Gutiérrez-Fonseca, PhD

BIOL-4994 & BIOL-4991 | BIOL-6994 & BIOL-6997
Fall 2025
2025-Aug-01 (updated: 2025-Sep-18)


dplyr
www.rstudio.com
summarize
mutate
select
filter

Data Wrangling with `dplyr`

- Over the past two weeks, we introduced several R functions for working with data.
- These included `colnames()`, `str()`, `summary()`, `cbind()`, and others.
- These are **base R functions**, and learning a few common ones will serve you well.

Data Wrangling with `dplyr`

- Another set of functions for data wrangling comes from the package `dplyr`.
- `dplyr` is part of the tidyverse and is designed to make data manipulation easier and more readable.
- The `dplyr` package has its own website, and a complete tutorial is available in Chapter 5 of the ebook R for Data Science by Garrett Grolemund and Hadley Wickham.

The `dplyr` package

- `dplyr` is a grammar of data manipulation, providing a consistent set of **verbs** to handle common data tasks.

The `dplyr` package

- We will cover six of the most commonly used dplyr functions, as well as the pipe operator `%>%`, which lets you combine functions smoothly.

- Rows:
  - `filter()` → keep rows matching conditions
  - `arrange()` → reorder rows
  - `slice()` → select rows by position
- Columns:
  - `select()` → pick columns by name or position
  - `reame()` → rename columns
  - `mutate()` → create new columns

- Groups
  - `group_by()` → define groups
  - `summarize()` → collapse data within groups
  - `group_by()` + `summarise()` → grouped summaries
  - `count()` → quick group counts
- `%>%` (shortcut: Ctrl + Shift + M) → “then”

`dplyr`: the pipe `%>%`

- All dplyr functions take a data frame as the first argument.
- Without the pipe, you'd need nested calls or temporary objects.
- With %>%, you can write operations left to right, top to bottom:

```
# Without pipe (nested calls)
f(g(h(data)))
```

```
# With pipe (easier to read)
data %>%
  h() %>%
  g() %>%
  f()
```

`dplyr`: the pipe `%>%`

- The pipe can be read as “then”.
- It passes the result of one function into the next.
- This makes your code:
  - Easier to read (step by step).
  - Easier to write (no need for temporary objects).
  - Easier to debug (you can run each step separately).

```
data %>%
  step_one() %>%      # then
  step_two() %>%      # then
  step_three()
```

`dplyr`: the pipe `%>%` vs `|>`

- Both %>% and |> are pipe operators: they pass the result of one expression into the next.
- `%>%`
  - Comes from the magrittr package (loaded with dplyr).
  - Supports placeholders like . for more flexibility.
  - Shortcut: Ctrl + Shift + M.
- `|>`
  - The base R pipe (introduced in R 4.1).
  - No extra package needed.

The `dplyr` package

- If you haven't installed dplyr yet, do so with:

```
install.packages('dplyr')
```

- Then load the package with:

```
library("dplyr")
```

Example dataset

```
head(starwars)
```

```
## # A tibble: 6 × 14
##   name      height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Luke Sky…    172    77 blond      fair       blue             19   male
## 2 C-3PO        167    75 <NA>       gold       yellow          112   none
## 3 R2-D2         96    32 <NA>       white, bl… red              33   none
## 4 Darth Va…    202   136 none       white      yellow           41.9 male
## 5 Leia Org…    150    49 brown      light      brown            19   fema…
## 6 Owen Lars    178   120 brown, gr… light      blue             52   male
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: filter()

- `filter()` selects a subset of rows in a data frame or tibble.
- The first argument is always the data frame.
- The second (and subsequent) arguments are conditions on variables, keeping only rows where the condition is TRUE.

```
starwars %>%
  filter(species == "Droid")
```

```
## # A tibble: 6 × 14
##   name   height  mass hair_color skin_color  eye_color birth_year sex
##   <chr>   <int> <dbl> <chr>      <chr>       <chr>           <dbl> <chr>
## 1 C-3PO     167    75 <NA>       gold        yellow            112 none
## 2 R2-D2      96    32 <NA>       white, blue red                33 none
## 3 R5-D4      97    32 <NA>       white, red  red                NA none
## 4 IG-88     200   140 none       metal       red                15 none
## 5 R4-P17     96    NA none       silver, red red, blue          NA none
## 6 BB8        NA    NA none       none        black              NA none
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
```

R Base Equivalent

- The same operation can be done in base R using `subset()`:

```
subset(starwars, species == "Droid")
```

```
## # A tibble: 6 × 14
##   name   height  mass hair_color skin_color  eye_color birth_year sex
##   <chr>   <int> <dbl> <chr>      <chr>       <chr>           <dbl> <chr>
## 1 C-3PO     167    75 <NA>       gold        yellow            112 none
## 2 R2-D2      96    32 <NA>       white, blue red                33 none
## 3 R5-D4      97    32 <NA>       white, red  red                NA none
## 4 IG-88     200   140 none       metal       red                15 none
## 5 R4-P17     96    NA none       silver, red red, blue          NA none
## 6 BB8        NA    NA none       none        black              NA none
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: filter()

- You can filter rows using multiple conditions separated by a comma (acts as AND):

```
starwars %>%
  filter(skin_color == "light", eye_color == "brown")
```

```
## # A tibble: 7 × 14
##   name      height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Leia Org…    150    49 brown      light      brown              19 fema…
## 2 Biggs Da…    183    84 black      light      brown              24 male
## 3 Padmé Am…    185    45 brown      light      brown              46 fema…
## 4 Cordé        157    NA brown      light      brown              NA <NA>
## 5 Dormé        165    NA brown      light      brown              NA fema…
## 6 Raymus A…    188    79 brown      light      brown              NA male
## 7 Poe Dame…     NA    NA brown      light      brown              NA male
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: filter()

- You can filter rows using multiple conditions separated by a `&`:

```
filter(starwars, skin_color == "light" & eye_color == "brown")
```

```
## # A tibble: 7 × 14
##   name      height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Leia Org…    150    49 brown      light      brown              19 fema…
## 2 Biggs Da…    183    84 black      light      brown              24 male
## 3 Padmé Am…    185    45 brown      light      brown              46 fema…
## 4 Cordé        157    NA brown      light      brown              NA <NA>
## 5 Dormé        165    NA brown      light      brown              NA fema…
## 6 Raymus A…    188    79 brown      light      brown              NA male
## 7 Poe Dame…     NA    NA brown      light      brown              NA male
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: arrange()

- `arrange()` reorders the rows of a data frame or tibble.
- The first argument is the data frame, followed by one or more column names (or expressions) to order by.
- If multiple columns are provided, later columns are used to break ties in earlier ones.

```
starwars %>% arrange(height, mass)
```

```
## # A tibble: 87 × 14
##    name     height  mass hair_color skin_color eye_color birth_year sex
##    <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
##  1 Yoda         66    17 white      green      brown             896 male
##  2 Ratts T…     79    15 none       grey, blue unknown            NA male
##  3 Wicket …     88    20 brown      brown      brown               8 male
##  4 Dud Bolt     94    45 none       blue, grey yellow             NA male
##  5 R2-D2        96    32 <NA>       white, bl… red                33 none
##  6 R4-P17       96    NA none       silver, r… red, blue          NA none
##  7 R5-D4        97    32 <NA>       white, red red                NA none
##  8 Sebulba     112    40 none       grey, red  orange             NA male
```

dplyr: arrange()

- Use `desc()` to sort a column in descending order:

```
starwars %>% arrange(desc(height))
```

```
## # A tibble: 87 × 14
##    name     height  mass hair_color skin_color eye_color birth_year sex
##    <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
##  1 Yarael …    264    NA none       white      yellow           NA   male
##  2 Tarfful     234   136 brown      brown      blue             NA   male
##  3 Lama Su     229    88 none       grey       black            NA   male
##  4 Chewbac…    228   112 brown      unknown    blue            200   male
##  5 Roos Ta…    224    82 none       grey       orange           NA   male
##  6 Grievous    216   159 none       brown, wh… green, y…        NA   male
##  7 Taun We     213    NA none       grey       black            NA   fema…
##  8 Rugor N…    206    NA none       green      orange           NA   male
##  9 Tion Me…    206    80 none       grey       black            NA   male
## 10 Darth V…    202   136 none       white      yellow          41.9 male
## # ℹ 77 more rows
```

dplyr: slice()

- `slice()` selects rows by their position (row numbers).
- You can use it to select, remove, or duplicate rows.
- Example: selecting rows 5 through 10.

```
starwars %>% slice(5:10)
```

```
## # A tibble: 6 × 14
##   name      height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Leia Org…    150    49 brown      light      brown              19 fema…
## 2 Owen Lars    178   120 brown, gr… light      blue               52 male
## 3 Beru Whi…    165    75 brown      light      blue               47 fema…
## 4 R5-D4         97    32 <NA>       white, red red                NA none
## 5 Biggs Da…    183    84 black      light      brown              24 male
## 6 Obi-Wan …    182    77 auburn, w… fair       blue-gray          57 male
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: slice()

- slice() selects rows by their position (row numbers).
- It comes with several helper functions for common use cases:
  - slice_head() → select the first n rows
  - slice_tail() → select the last n rows

```
starwars %>%
  slice_head(n = 3)   # first 3 rows
```

```
## # A tibble: 3 × 14
##   name      height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Luke Sky…    172    77 blond      fair       blue               19 male
## 2 C-3PO        167    75 <NA>       gold       yellow            112 none
## 3 R2-D2         96    32 <NA>       white, bl… red                33 none
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: select()

- Datasets often have many columns, but you may only need a few of them.
- `select()` makes it easy to extract specific columns by name (or with helper functions).

```
# Select columns by name
starwars %>%
  select(hair_color, skin_color, eye_color)
```

```
## # A tibble: 87 × 3
##    hair_color    skin_color  eye_color
##    <chr>         <chr>       <chr>
##  1 blond         fair        blue
##  2 <NA>          gold        yellow
##  3 <NA>          white, blue red
##  4 none          white       yellow
##  5 brown         light       brown
##  6 brown, grey   light       blue
##  7 brown         light       blue
##  8 <NA>          white, red  red
```

dplyr: select()

- You can also select a range of columns using `:`

```
# Select all columns between hair_color and eye_color (inclusive)
starwars %>% select(hair_color:eye_color)
```

```
## # A tibble: 87 × 3
##    hair_color    skin_color  eye_color
##    <chr>         <chr>       <chr>
##  1 blond         fair        blue
##  2 <NA>          gold        yellow
##  3 <NA>          white, blue red
##  4 none          white       yellow
##  5 brown         light       brown
##  6 brown, grey   light       blue
##  7 brown         light       blue
##  8 <NA>          white, red  red
##  9 black         light       brown
## 10 auburn, white fair        blue-gray
```

dplyr: select()

- Use `!` to deselect (drop) columns:

```
# Select all columns except those from hair_color to eye_color (inclusive)
starwars %>% select(!(hair_color:eye_color))
```

```
## # A tibble: 87 × 11
##    name      height  mass birth_year sex   gender homeworld species films
##    <chr>      <int> <dbl>      <dbl> <chr> <chr>  <chr>     <chr>   <lis>
##  1 Luke Sky…    172    77       19   male  mascu… Tatooine  Human   <chr>
##  2 C-3PO        167    75      112   none  mascu… Tatooine  Droid   <chr>
##  3 R2-D2         96    32       33   none  mascu… Naboo     Droid   <chr>
##  4 Darth Va…    202   136       41.9 male  mascu… Tatooine  Human   <chr>
##  5 Leia Org…    150    49       19   fema… femin… Alderaan  Human   <chr>
##  6 Owen Lars    178   120       52   male  mascu… Tatooine  Human   <chr>
##  7 Beru Whi…    165    75       47   fema… femin… Tatooine  Human   <chr>
##  8 R5-D4         97    32       NA   none  mascu… Tatooine  Droid   <chr>
##  9 Biggs Da…    183    84       24   male  mascu… Tatooine  Human   <chr>
## 10 Obi-Wan …    182    77       57   male  mascu… Stewjon   Human   <chr>
```

dplyr: select()

- We can use `-` to extract all except particular column(s). For example, to drop the **hair_color** and **eye_color columns**:

```
# Select all columns except those from hair_color to eye_color (inclusive)
starwars %>% select(-hair_color, -eye_color)
```

```
## # A tibble: 87 × 12
##    name          height  mass skin_color birth_year sex   gender homeworld
##    <chr>          <int> <dbl> <chr>           <dbl> <chr> <chr>  <chr>
##  1 Luke Skywal…    172    77 fair             19   male  mascu… Tatooine
##  2 C-3PO           167    75 gold            112   none  mascu… Tatooine
##  3 R2-D2            96    32 white, bl…       33   none  mascu… Naboo
##  4 Darth Vader     202   136 white           41.9 male  mascu… Tatooine
##  5 Leia Organa     150    49 light           19   fema… femin… Alderaan
##  6 Owen Lars       178   120 light           52   male  mascu… Tatooine
##  7 Beru Whites…    165    75 light           47   fema… femin… Tatooine
##  8 R5-D4            97    32 white, red      NA   none  mascu… Tatooine
##  9 Biggs Darkl…    183    84 light           24   male  mascu… Tatooine
```

dplyr: select()

- We can also specify column positions, or combine column positions and names:

```
# Select all columns except those from hair_color to eye_color (inclusive)
starwars %>% select(1, 2, hair_color, eye_color)
```

```
## # A tibble: 87 × 4
##    name               height hair_color    eye_color
##    <chr>               <int> <chr>         <chr>
##  1 Luke Skywalker        172 blond         blue
##  2 C-3PO                 167 <NA>          yellow
##  3 R2-D2                  96 <NA>          red
##  4 Darth Vader           202 none          yellow
##  5 Leia Organa           150 brown         brown
##  6 Owen Lars             178 brown, grey   blue
##  7 Beru Whitesun Lars    165 brown         blue
##  8 R5-D4                  97 <NA>          red
##  9 Biggs Darklighter     183 black         brown
## 10 Obi-Wan Kenobi        182 auburn, white blue-gray
```

dplyr: select() with contains()

To select only columns with names containing the letter 'D':

```
starwars %>% select(contains('c'))
```

```
## # A tibble: 87 × 5
##    hair_color    skin_color  eye_color species vehicles
##    <chr>         <chr>       <chr>     <chr>   <list>
##  1 blond         fair        blue      Human   <chr [2]>
##  2 <NA>          gold        yellow    Droid   <chr [0]>
##  3 <NA>          white, blue red       Droid   <chr [0]>
##  4 none          white       yellow    Human   <chr [0]>
##  5 brown         light       brown     Human   <chr [1]>
##  6 brown, grey   light       blue      Human   <chr [0]>
##  7 brown         light       blue      Human   <chr [0]>
##  8 <NA>          white, red  red       Droid   <chr [0]>
##  9 black         light       brown     Human   <chr [0]>
## 10 auburn, white fair        blue-gray Human   <chr [1]>
## # ℹ 77 more rows
```

dplyr: select() with starts_with()

- start_with() and ends_with() offer more specificity for select(). If we want all columns beginning with the letter ‘c’::

```
starwars %>% select(starts_with('n'))
```

```
## # A tibble: 87 × 1
##    name
##    <chr>
##  1 Luke Skywalker
##  2 C-3PO
##  3 R2-D2
##  4 Darth Vader
##  5 Leia Organa
##  6 Owen Lars
##  7 Beru Whitesun Lars
##  8 R5-D4
##  9 Biggs Darklighter
## 10 Obi-Wan Kenobi
```

dplyr: select() with ends_with()

- start_with() and ends_with() offer more specificity for select(). If we want all columns beginning with the letter ‘c’:

```
starwars %>% select(ends_with('d'))
```

```
## # A tibble: 87 × 1
##    homeworld
##    <chr>
##  1 Tatooine
##  2 Tatooine
##  3 Naboo
##  4 Tatooine
##  5 Alderaan
##  6 Tatooine
##  7 Tatooine
##  8 Tatooine
##  9 Tatooine
## 10 Stewjon
```

dplyr: rename()

- `rename()` changes column names without altering the data.

```
starwars %>%
  dplyr::rename(Character = name)
```

```
## # A tibble: 87 × 14
##    Character      height  mass hair_color skin_color eye_color birth_year
##    <chr>           <int> <dbl> <chr>      <chr>      <chr>           <dbl>
##  1 Luke Skywalker    172    77 blond      fair       blue             19
##  2 C-3PO             167    75 <NA>       gold       yellow          112
##  3 R2-D2              96    32 <NA>       white, bl… red              33
##  4 Darth Vader       202   136 none       white      yellow           41.9
##  5 Leia Organa       150    49 brown      light      brown            19
##  6 Owen Lars         178   120 brown, gr… light      blue             52
##  7 Beru Whitesun…    165    75 brown      light      blue             47
##  8 R5-D4              97    32 <NA>       white, red red              NA
##  9 Biggs Darklig…    183    84 black      light      brown            24
## 10 Obi-Wan Kenobi    182    77 auburn, w… fair       blue-gray        57
```

`dplyr: rename()`

- How would you rename the column name to Character using base R functions instead of dplyr?

dplyr: rename()

- How would you rename the column name to Character using base R functions instead of dplyr?

```
# Change "name" column to "Character"
colnames(starwars)[colnames(starwars) == "name"] <-"Character"
head(starwars,3)
```

```
## # A tibble: 3 × 14
##   Character height  mass hair_color skin_color eye_color birth_year sex
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr>
## 1 Luke Sky…    172    77 blond      fair       blue             19 male
## 2 C-3PO        167    75 <NA>       gold       yellow          112 none
## 3 R2-D2         96    32 <NA>       white, bl… red              33 none
## # ℹ 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
## #   films <list>, vehicles <list>, starships <list>
```

dplyr: mutate()

- Besides selecting existing columns, you can add **new columns** that are functions of existing ones.

```
newstarwars <- starwars %>% mutate(height_m = height / 100)
# Select the last 3 columns
newstarwars %>%
  select(height_m, height, everything())
```

```
## # A tibble: 87 × 15
##    height_m height Character          mass hair_color skin_color eye_color
##       <dbl>  <int> <chr>             <dbl> <chr>      <chr>      <chr>
##  1     1.72    172 Luke Skywalker       77 blond      fair       blue
##  2     1.67    167 C-3PO                75 <NA>       gold       yellow
##  3     0.96     96 R2-D2                32 <NA>       white, bl… red
##  4     2.02    202 Darth Vader         136 none       white      yellow
##  5     1.5     150 Leia Organa          49 brown      light      brown
##  6     1.78    178 Owen Lars           120 brown, gr… light      blue
##  7     1.65    165 Beru Whitesun L…     75 brown      light      blue
```

dplyr: mutate()

- `mutate()` creates new columns or transforms existing ones.
- The `.keep` argument controls which columns are kept in the output:

```
newstarwars %>%
  mutate(
    height_m = height / 100,
    .keep = "none")
```

```
## # A tibble: 87 × 1
##    height_m
##       <dbl>
##  1     1.72
##  2     1.67
##  3     0.96
##  4     2.02
##  5     1.5
##  6     1.78
##  7     1.65
```

dplyr: mutate() with ifelse()

- `ifeslse` is often used inside `mutate()` to create new **categorical** variable.
- It test a logical condition for each row and returns:
  - one value if the condition is **TRUE**
  - another value if the condition is **FALSE**
- Syntax: `ifelse(condition, value_if_true, value_if_false)`

```
starwars %>%
  mutate(height_cat = ifelse(height > 100, "tall", "small"))
```

```
## # A tibble: 87 × 15
##    Character      height  mass hair_color skin_color eye_color birth_year
##    <chr>           <int> <dbl> <chr>      <chr>      <chr>           <dbl>
##  1 Luke Skywalker    172    77 blond      fair       blue             19
##  2 C-3PO             167    75 <NA>       gold       yellow          112
##  3 R2-D2              96    32 <NA>       white, bl… red              33
##  4 Darth Vader       202   136 none       white      yellow           41.9
##  5 Leia Organa       150    49 brown      light      brown            19
##  6 Owen Lars         178   120 brown, gr… light      blue             52
```

dplyr: mutate() with ifelse()

- We can also combine multiple dplyr functions in a single pipeline:

```
starwars %>%
  mutate(height_cat = ifelse(height > 100, "tall", "small")) %>%
   select(height, height_cat, everything())
```

```
## # A tibble: 87 × 15
##    height height_cat Character       mass hair_color skin_color eye_color
##     <int> <chr>      <chr>          <dbl> <chr>      <chr>      <chr>
##  1    172 tall       Luke Skywalker    77 blond      fair       blue
##  2    167 tall       C-3PO             75 <NA>       gold       yellow
##  3     96 small      R2-D2             32 <NA>       white, bl… red
##  4    202 tall       Darth Vader      136 none       white      yellow
##  5    150 tall       Leia Organa       49 brown      light      brown
##  6    178 tall       Owen Lars        120 brown, gr… light      blue
##  7    165 tall       Beru Whitesun…    75 brown      light      blue
##  8     97 small      R5-D4             32 <NA>       white, red red
##  9    183 tall       Biggs Darklig…    84 black      light      brown
```

dplyr: mutate() with ifelse() and ggplot2

- After creating a new variable, we can quickly visualize it with ggplot2:

R Code | Plot

```
starwars %>%
mutate(height_cat = ifelse(height > 100, "tall", "small")) %>%
ggplot(aes(x=height, fill= height_cat )) +
geom_histogram()
```

```
## stat_bin() using bins = 30. Pick better value with binwidth.
```

`dplyr: summarise(), mean()`

- `summarise()` collapses a data frame to a single row (or one row per group if grouped).
- Useful for computing summary statistics:
- `na.rm = TRUE` ignores missing values in calculations.

```
starwars %>% summarise(height = mean(height, na.rm = TRUE))
```

```
## # A tibble: 1 × 1
##   height
##    <dbl>
## 1   175.
```

`dplyr: summarise(), mean(), min(), max()`

- `summarise()` collapses a data frame to a single row (or one row per group if grouped).
- Useful for computing summary statistics:
- `na.rm = TRUE` ignores missing values in calculations.

```
starwars %>% summarise(height = mean(height, na.rm = TRUE),
                       min= min(height, na.rm = TRUE),
                       max= max(height, na.rm = TRUE))
```

```
## # A tibble: 1 × 3
##   height   min   max
##    <dbl> <dbl> <dbl>
## 1   175.   175.   175.
```

dplyr: group_by(), str()

- Exploring your data with `str()`

```
starwars %>% str()
```

```
## tibble [87 × 14] (S3: tbl_df/tbl/data.frame)
##  $ Character : chr [1:87] "Luke Skywalker" "C-3PO" "R2-D2" "Darth Vader" ...
##  $ height    : int [1:87] 172 167 96 202 150 178 165 97 183 182 ...
##  $ mass      : num [1:87] 77 75 32 136 49 120 75 32 84 77 ...
##  $ hair_color: chr [1:87] "blond" NA NA "none" ...
##  $ skin_color: chr [1:87] "fair" "gold" "white, blue" "white" ...
##  $ eye_color : chr [1:87] "blue" "yellow" "red" "yellow" ...
##  $ birth_year: num [1:87] 19 112 33 41.9 19 52 47 NA 24 57 ...
##  $ sex       : chr [1:87] "male" "none" "none" "male" ...
##  $ gender    : chr [1:87] "masculine" "masculine" "masculine" "masculine" ...
##  $ homeworld : chr [1:87] "Tatooine" "Tatooine" "Naboo" "Tatooine" ...
##  $ species   : chr [1:87] "Human" "Droid" "Droid" "Human" ...
##  $ films     :List of 87
##   ..$ : chr [1:5] "A New Hope" "The Empire Strikes Back" "Return of the Jedi" "Revenge of
```

`dplyr`: `group_by()`, `summarise()`

- `group_by()` defines groups for group-wise operations.
- `summarise()` then computes summary statistics for each group.

```
starwars %>%
  group_by(species) %>%
  summarise(
    mean_height = mean(height, na.rm = TRUE),
    sd_height = sd(height, na.rm = TRUE)
  )
```

```
## # A tibble: 38 × 3
##    species   mean_height sd_height
##    <chr>           <dbl>     <dbl>
##  1 Aleena             79        NA
##  2 Besalisk          198        NA
##  3 Cerean            198        NA
##  4 Chagrian          196        NA
##  5 Clawdite          168        NA
```

`dplyr: group_by(), summarise()`

- `group_by()` defines groups for group-wise operations.
- `summarise()` then computes summary statistics for each group.

```
starwars %>%
 group_by(homeworld, gender) %>%
   summarise(mean_height_by_gender = mean(height, na.rm = TRUE))
```

```
## # A tibble: 57 × 3
## # Groups:   homeworld [49]
##    homeworld      gender    mean_height_by_gender
##    <chr>          <chr>                     <dbl>
##  1 Alderaan       feminine                  150
##  2 Alderaan       masculine                 190.
##  3 Aleen Minor    masculine                  79
##  4 Bespin         masculine                 175
##  5 Bestine IV     <NA>                      180
##  6 Cato Neimoidia masculine                 191
##  7 Cerea          masculine                 198
```

dplyr: count()

- A very common operation is to count the number of observations for each group.
- `count()` takes the name of one or more columns that contain the groups we are interested in, and we can optionally sort the results in descending order by adding `sort=TRUE`.

```
starwars %>%
    group_by(species) %>%
    count(homeworld, sort = TRUE)
```

```
## # A tibble: 57 × 3
## # Groups:   species [38]
##    species  homeworld     n
##    <chr>    <chr>     <int>
##  1 Human    Tatooine      8
##  2 Human    <NA>          6
##  3 Human    Naboo         5
##  4 Droid    <NA>          3
##  5 Gungan   Naboo         3
##  6 Human    Alderaan      3
```

dplyr: sample_n()

- `sample_n()` randomly selects a specified number of rows from a data frame.
- Useful for exploring a subset of a large dataset.

```
starwars %>% sample_n(10)
```

```
## # A tibble: 10 × 14
##    Character      height  mass hair_color skin_color eye_color birth_year
##    <chr>           <int> <dbl> <chr>      <chr>      <chr>           <dbl>
##  1 Roos Tarpals      224    82 none       grey       orange             NA
##  2 Padmé Amidala     185    45 brown      light      brown              46
##  3 Greedo            173    74 <NA>       green      black              44
##  4 Yarael Poof       264    NA none       white      yellow             NA
##  5 Cliegg Lars       183    NA brown      fair       blue               82
##  6 Kit Fisto         196    87 none       green      black              NA
##  7 Jar Jar Binks     196    66 none       orange     orange             52
##  8 Raymus Antill…    188    79 brown      light      brown              NA
##  9 Finn               NA    NA black      dark       dark               NA
## 10 Rugor Nass        206    NA none       green      orange             NA
```

`dplyr` recap

- All **verbs** in `dplyr` share a consistent grammar:
  - The **first argument** is always a data frame (or tibble).
  - The **next arguments** describe what to do with the columns.
  - You can refer to columns **directly** (no `$` needed).
  - The result is always a **new data frame (tibble)**.